

Clancy Space | Editorial Personal Operating System

完整产品框架与架构规范 v1.0

0. 文档说明

0.1 文档目的

本文档用于完整定义 Clancy Space 的产品框架，帮助设计与开发工具理解：

- 这个网站究竟是什么
- 它服务于哪些用户
- 它解决什么问题
- 为什么不能只做成一个高级 Landing Page
- “控制台 / Personal OS” 具体意味着什么
- 电影化入口与控制台之间如何衔接
- 项目、案例、实验、方法论和笔记如何组织
- 内容与模板如何解耦
- 后续如何持续更新
- 第一阶段和未来阶段分别应该实现什么

本文档的优先级高于此前单纯以页面 Section 为中心的执行计划。

0.2 当前问题

现有初稿虽然在视觉上接近 V3 / V4：

- 深色电影感背景
- 粒子系统
- Orbit 轨道
- Editorial Typography
- Scroll Motion
- 玻璃质感组件

但产品结构仍然接近：

```
Hero
↓
项目展示
↓
理念介绍
```

↓
联系入口

这类结构本质上仍是：

一个经过高级视觉包装的个人 Landing Page。

它缺少原始构想中的核心产品层：

Personal Workspace / Portfolio OS / 个人控制台。

因此，当前要解决的不是“补一个 Dock”或“补一个控制台组件”，而是重新建立网站的完整产品架构。

1. 产品名称与定义

1.1 暂定名称

主名称：

Clancy Space

产品概念名称：

Editorial Personal Operating System

内部概念名称可使用：

- ClancyOS
- Personal Workspace
- Portfolio OS
- Personal System
- Digital Studio

不建议在所有面向用户的文案中高频使用“OS”，避免过度产品化和概念化。

1.2 一句话定义

Clancy Space 是一个结合电影化个人入口、Editorial Motion 视觉语言、Workspace 控制台交互和持续内容系统的个人数字空间。

1.3 完整定义

Clancy Space 不是传统简历网站，也不是产品宣传页。

它是一个持续承载以下内容的个人系统：

- 个人定位
- 项目作品
- 产品案例
- AI 实验
- Agent Workflow
- 产品方法论
- 技术研究
- 笔记与文章
- 当前构建状态
- 职业发展与联系方式

用户进入网站后，不只是“阅读一个页面”，而是进入一个可以探索、搜索、切换和调用内容的个人工作空间。

2. 产品愿景

2.1 长期愿景

建立一个能够持续承载个人项目、方法论、研究、实验与职业品牌的数字操作系统。

它应该随着个人能力与项目积累而持续生长，而不是制作完成后长期不更新。

2.2 用户最终感知

访问者离开网站时，应该形成以下认知：

这是一个真实的人，而不是某个 AI 产品品牌。

他不仅做过一些项目，
还形成了自己的产品思考、工作流和系统方法。

他不是只会调用 AI 工具，
而是在研究如何把 AI 产品化、流程化、系统化。

这个网站本身也体现了他的产品能力。

3. 核心问题

3.1 普通个人网站的问题

传统个人网站通常采用：

自我介绍
项目列表
工作经历
技能
联系方式

这类结构的问题是：

- 与简历重复
- 缺乏探索感
- 难以展现思考过程
- 项目之间没有关系
- 无法表达持续构建状态
- 很难承载方法论与研究
- 后续内容越来越多时容易失控

3.2 普通作品集的问题

普通作品集通常将项目包装为单独卡片：

项目名称
一句话简介
功能介绍
截图
查看详情

这容易导致：

- 项目像产品广告
- 看不到个人角色
- 看不到关键判断
- 看不到方法论
- 看不到失败和复盘
- 看不到项目之间的能力积累

3.3 纯电影化网站的问题

V3 / V4 风格虽然有高级感，但如果只保留：

抽象标题
电影化 Hero
粒子动效

产品卡片
理念 Quote

会产生两个问题：

1. 用户不知道这是个人网站。
2. 用户会把它误解为 AI 产品或创意工作室宣传页。

3.4 传统控制台的问题

如果直接做成控制台或 Dashboard，又容易出现：

- 后台管理系统感
- SaaS 产品感
- 信息密度过高
- 缺乏情绪和个人气质
- 移动端体验差
- 过度模仿 macOS
- 形式大于内容

4. 核心解决方案

最终产品不在“电影化网站”和“控制台”之间二选一。

应采用：

Cinematic Arrival + Personal Workspace

即双模式架构。

Arrival Mode
电影化个人入口

↓

Transition
从观看转向探索

↓

Workspace Mode
个人控制台与内容系统

5. 产品目标

5.1 一级目标

目标 A：快速建立个人认知

用户进入网站 5 秒内应知道：

- 这是一个个人网站
 - 这个人是谁
 - 他关注什么
 - 这里可以看到什么
-

目标 B：展示项目之外的能力

网站不仅展示“做了什么”，还需要展示：

- 如何判断问题
 - 如何拆解需求
 - 如何做产品决策
 - 如何设计 AI Workflow
 - 如何建立评测与治理机制
 - 如何进行复盘
-

目标 C：形成持续更新的内容系统

后续新增以下内容时，不需要重做网页：

- 新项目
 - 新案例
 - 新笔记
 - 新实验
 - 新方法论
 - 当前状态更新
-

目标 D：让网站本身成为作品

网站需要体现：

- 产品架构能力
 - 信息架构能力
 - 交互设计能力
 - 内容系统设计能力
 - AI Coding 协作能力
 - 长期产品思维
-

5.2 二级目标

- 支持招聘与面试场景
 - 支持合作方快速了解
 - 支持 GitHub 项目导流
 - 支持文章和研究沉淀
 - 支持项目成熟度分层
 - 支持未来中英文切换
 - 支持未来可视化编辑
-

6. 非目标

第一阶段不应把项目做成：

- 完整社交平台
 - 博客 CMS
 - 项目管理软件
 - 真正的桌面操作系统
 - 数据分析后台
 - 复杂 3D 游戏
 - 多用户系统
 - 需要登录的管理系统
 - 仿 macOS 桌面
 - 纯视觉炫技实验
-

7. 目标用户

7.1 招聘方与面试官

核心需求：

- 快速知道候选人是谁
- 快速理解其定位
- 查看代表性项目
- 确认候选人真实参与程度
- 判断产品思考和系统能力
- 获取简历、GitHub 和联系方式

行为特点：

- 浏览时间可能较短
 - 不一定探索完整网站
 - 需要明确导航
 - 需要快速路径
-

7.2 AI 产品经理与技术同行

核心需求：

- 查看具体项目
- 理解方法论
- 查看 Agent / RAG / Workflow 设计
- 阅读架构、评测和治理内容
- 寻找可复用思路

行为特点：

- 愿意深度浏览
 - 关注技术细节与取舍
 - 可能通过搜索进入具体内容
-

7.3 潜在合作方

核心需求：

- 判断能力边界
 - 了解项目经验
 - 了解关注方向
 - 判断是否适合合作
 - 快速联系
-

7.4 普通访问者

核心需求：

- 快速获得清晰印象
 - 理解网站如何使用
 - 不被复杂控制台劝退
-

8. 核心产品原则

8.1 优雅地清晰

清晰 ≠ 简历式直白
优雅 ≠ 抽象到无法理解

实现方式：

- 主标题负责表达气质

- 副标题负责解释身份
 - Identity Strip 负责确认领域
 - Workspace 负责承载深度内容
-

8.2 先建立认知，再允许探索

不能一打开就让用户面对复杂控制台。

正确顺序是：

认识这个人
↓
理解他的方向
↓
产生继续探索的兴趣
↓
进入 Workspace

8.3 控制台是交互模型，不是视觉皮肤

控制台的核心不是：

- 窗口
- Dock
- 系统图标

而是：

- 内容被组织为应用
 - 用户可以打开、关闭和切换
 - 支持搜索与命令
 - 支持返回和历史
 - 支持状态展示
 - 支持持续更新
-

8.4 项目展示个人判断，而不是产品卖点

项目描述优先回答：

我为什么做
我解决了什么
我负责什么
我如何判断
我做了什么取舍

我如何验证
我学到了什么

而不是只写：

产品功能
产品优势
目标用户
功能模块

8.5 内容与表现分离

所有内容必须由配置或 Markdown 数据驱动。

组件不能写死：

- 项目名称
- 文章标题
- 标签
- 状态
- 描述
- 外部链接
- 当前关注方向

8.6 动效服务于空间与认知

所有动效必须服务于：

- 引导注意力
- 建立层级
- 反馈操作
- 表达内容关系
- 制造空间感

禁止为了“高级感”堆积无意义动画。

9. 产品总体架构

整个产品分为五层。

Clancy Space
|
|—— 1. Arrival Layer

- | └—— 负责建立身份认知与第一印象
- |
- |—— 2. Workspace Shell
- | └—— 负责全局导航、状态、搜索和内容容器
- |
- |—— 3. Content Apps
- | └—— 负责项目、案例、实验、方法论、笔记等内容
- |
- |—— 4. Content Engine
- | └—— 负责内容数据、分类、标签、关系和更新
- |
- |—— 5. Experience System
- | └—— 负责视觉、动效、响应式、性能与无障碍

10. 第一层：Arrival Layer

10.1 作用

Arrival Layer 是电影化个人入口。

它负责：

- 建立情绪
- 建立身份
- 表达关注方向
- 解释网站是什么
- 引导用户进入 Workspace

10.2 Arrival Layer 结构

Arrival Layer

- |—— Navigation
- |—— Hero
- |—— Identity Statement
- |—— Focus Strip
- |—— Current Status Preview
- |—— Enter Workspace CTA
- |—— Visual Environment

10.3 Hero

内容结构

Eyebrow
Main Statement
Personal Identity
Supporting Description
Primary CTA
Secondary CTA

推荐表达

AI PRODUCT / AGENT WORKFLOW / SYSTEMS

构建 AI 产品，
也构建让 AI 落地的系统。

我是 Clancy，关注 AI 产品、Agent Workflow、
RAG 应用与复杂任务自动化。

这个空间记录我的项目实验、产品方法论、
系统设计与持续构建过程。

CTA

进入工作空间
查看代表项目

10.4 Identity Strip

目的：

明确网站主体是一个人，而不是一个产品。

示例：

AI 产品
Agent Workflow
RAG 与知识系统
产品发现
评测与治理
自动化情报

10.5 Current Status Preview

首页可用极轻量形式展示：

CURRENTLY BUILDING
Personal Portfolio OS

CURRENTLY EXPLORING
Agent Evaluation & Memory Governance

此处仅作预览，完整状态位于 Workspace。

10.6 视觉环境

保留 V3 / V4 的核心视觉：

- 深石墨背景
- 暖白文字
- 鼠尾草绿强调色
- 粒子场
- Orbit 轨道
- Mouse Light
- Scroll Parallax
- Editorial Typography
- 大面积留白

11. Arrival → Workspace 转换

11.1 转换触发方式

允许两种方式：

方式 A：点击进入

用户点击：

进入工作空间

页面执行转场。

方式 B：滚动进入

当用户滚动通过 Arrival Layer 后，Workspace Shell 逐渐显现。

11.2 转场逻辑

Hero 内容降低透明度
↓
Orbit 向 Workspace 背景收拢
↓
粒子连接密度轻微变化
↓
Dock 出现
↓
Workspace Header 出现
↓
默认 Home App 打开

11.3 转场原则

- 过渡时间不宜过长
- 不应播放不可跳过的动画
- 用户滚动必须始终可控
- Reduced Motion 模式下取消复杂转场
- 返回顶部时可重新进入 Arrival 状态

12. 第二层：Workspace Shell

12.1 定义

Workspace Shell 是整个网站的全局控制层。

它不承载具体内容，而负责：

- 导航
- 内容打开
- 状态切换
- 搜索
- 命令
- 路径
- 返回
- 当前应用状态
- 内容容器

12.2 Workspace Shell 组成

Workspace Shell

- └── Workspace Header
- └── Global Dock
- └── Command Palette
- └── Active App Indicator
- └── Workspace Status
- └── Main Content Panel
- └── Breadcrumb / Back Navigation
- └── Optional Side Context
- └── Background Experience

12.3 Workspace Header

建议包含：

Clancy Space

当前应用名称

当前路径

⌘K 搜索

状态指示

要求：

- 高度低
- 弱边框
- 半透明
- 不像后台系统顶部栏

12.4 Global Dock

作用

Global Dock 是主要内容入口。

建议 App：

Home

Profile

Projects

Cases

Lab

Thinking

Notes
Resume
Contact

行为

- Workspace 模式下出现
- 桌面端可位于底部悬浮
- 鼠标靠近时增强
- 当前 App 有弱高亮
- hover 显示文字
- 支持键盘导航
- 移动端转为底部导航或 Menu Sheet

禁止

- 仿 macOS 彩色图标
- 过度缩放
- 复杂弹跳
- 拟物 Dock
- 图标无文字

12.5 Command Palette

快捷键

⌘K
Ctrl+K
/

支持的操作

打开 Projects
查看 FitCheck
搜索 Agent
打开最新笔记
查看简历
查看正在构建
联系 Clancy
返回首页
切换语言

搜索范围

- App
- 项目
- 案例

- 方法论
- 笔记
- 标签
- 外部链接

结果结构

类型
标题
摘要
所属模块
匹配标签
操作

12.6 Active App Indicator

用于告诉用户当前处于哪个模块。

示例：

WORKSPACE / PROJECTS
WORKSPACE / THINKING
WORKSPACE / NOTES

表现应克制，不做浏览器标签页风格。

12.7 Workspace Status

展示网站和个人当前状态。

建议字段：

Current Focus
Currently Building
Recently Updated
Availability
Last Updated

示例：

CURRENTLY BUILDING
Clancy Space

CURRENTLY EXPLORING

Agent Runtime / Evaluation / Memory Governance

LATEST UPDATE

Zero-to-One Testing Framework

AVAILABILITY

Open to AI Product opportunities

12.8 Main Content Panel

内容 App 的统一容器。

要求：

- 全宽或宽内容面板
- 保留足够留白
- 支持内部滚动或页面滚动
- 支持打开详情
- 支持返回 Workspace
- 不允许多个自由浮动窗口互相覆盖

12.9 Panel System，而不是桌面窗口系统

建议使用：

Controlled Panel System

而不是：

Freeform Desktop Window System

原因：

- 可读性更好
- 移动端更容易适配
- 路由更清晰
- 状态更容易管理
- 不会变成 macOS 模仿秀

可以保留：

- Window-like Header
- Panel Depth
- Layered Glass
- Open / Close Transition

但不做：

- 任意拖动
- 任意缩放
- 多窗口重叠
- 桌面图标排列

13. Workspace 状态模型

建议定义以下核心状态：

```
type ExperienceMode = "arrival" | "workspace";

type WorkspaceApp =
  | "home"
  | "profile"
  | "projects"
  | "cases"
  | "lab"
  | "thinking"
  | "notes"
  | "resume"
  | "contact";

type WorkspaceState = {
  mode: ExperienceMode;
  activeApp: WorkspaceApp;
  activeItemId?: string;
  commandPaletteOpen: boolean;
  navigationHistory: NavigationEntry[];
  searchQuery: string;
};
```

13.1 核心状态转换

```
Arrival
→ Enter Workspace
→ Workspace Home

Workspace Home
→ Open App
→ App Overview

App Overview
→ Open Item
```

→ Item Detail

Item Detail

→ Back

→ App Overview

Any State

→ Command Palette

→ Open Target

Any Workspace State

→ Home / Logo

→ Workspace Home

Workspace

→ Return to Arrival

→ Hero

14. 第三层：Content Apps

Content Apps 是 Workspace 中的内容模块。

15. Home App

15.1 作用

Workspace 默认主页。

它不是另一个 Hero，而是控制台摘要。

15.2 内容结构

Welcome / Identity Summary

Current Status

Featured Work

Latest Update

Recent Notes

Current Experiments

Quick Actions

15.3 Quick Actions

查看代表项目
阅读方法论
打开最新笔记
下载简历
联系我

16. Profile App

16.1 作用

介绍个人身份、经历、动机和能力方向。

16.2 内容建议

Who I Am
What I Care About
How I Work
Career Direction
Selected Experience
Core Capabilities
Tools & Methods

16.3 表达原则

避免传统简历式：

熟练掌握……
具备较强……
精通……

优先使用：

我关注什么
我如何做判断
我在哪些问题上持续投入
哪些经历影响了我的产品选择

17. Projects App

17.1 作用

展示所有具有独立产品或工具形态的项目。

17.2 项目成熟度分层

必须支持：

MATURE / SHIPPED

成熟或已交付

BUILDING

正在持续构建

PROTOTYPE

原型验证

RESEARCH

研究型项目

ARCHIVED

已停止或归档

17.3 推荐分组

结合当前项目：

MATURE / SHIPPED

- Prompix

- Peel

BUILDING

- AI Trend Radar

- Zero-to-One Product Discovery

- Repo-Harness

- FitCheck

EXPERIMENTS

- AI PRD Generator

- Industry Intelligence Dashboard

- 其他短期原型

具体分组应由配置文件决定，不写死在组件中。

17.4 项目列表视图

每个项目应展示：

项目名称
一句话问题定义
个人角色
项目状态
核心能力标签
更新时间
详情入口
GitHub / Demo

17.5 项目详情结构

01. 项目摘要
02. 问题背景
03. 为什么值得解决
04. 用户与场景
05. 我的角色
06. 约束条件
07. 产品判断
08. 方案结构
09. AI / 技术架构
10. 关键取舍
11. 测试与验证
12. 结果与证据
13. 失败与复盘
14. 当前状态
15. 下一阶段
16. Links / Demo / GitHub

18. Case Studies App

18.1 与 Projects 的区别

Projects = 我构建了什么
Cases = 我如何解决问题

一个 Project 可以关联多个 Case Study。

18.2 案例类型

Product Discovery
Workflow Design
Agent Architecture
RAG Optimization
Evaluation Design
Failure Recovery
UI / UX Redesign
Automation Pipeline

18.3 案例结构

Context
Problem
Constraints
Evidence
Decision
Alternatives Considered
Solution
Implementation
Validation
Outcome
Reflection

19. Lab App

19.1 定位

用于承载尚未成熟、但体现探索方向的实验。

19.2 内容类型

- Vibe Coding 实验
- Agent Workflow
- Prompt 实验
- API 原型
- 新模型测试
- 小型工具

- Benchmark
- 技术验证
- UI 动效 Demo
- 学习型项目

19.3 Lab 与 Projects 的区别

Projects

需要相对完整的问题、方案和成果。

Lab

允许未完成、短周期、失败和探索性内容。

19.4 实验卡片字段

实验名称
实验问题
假设
使用技术
实验结果
状态
结论
是否继续

20. Thinking App

20.1 定位

不是普通博客，而是：

Product and Systems Thinking Library

20.2 内容类型

- 产品方法论
- Agent 设计原则
- Workflow 设计
- Stage Gate
- Controller / Producer / Auditor
- Memory Governance
- Evaluation

- Benchmark
 - Product Discovery
 - Spec First / Plan Second
 - AI Coding 协作方法
-

20.3 内容组织

建议同时支持：

Topics
Frameworks
Principles
Playbooks
Essays

20.4 方法论条目结构

名称
解决的问题
适用场景
核心原则
步骤
输入
输出
风险
示例项目
相关笔记

21. Notes App

21.1 定位

Notes 是让网站持续生长的核心模块。

它需要支持高频、低成本更新。

21.2 内容类型

- 学习笔记
- 项目日志
- 技术记录

- 产品观察
 - 工具测试
 - 面试思考
 - 研究摘要
 - 每周回顾
 - 灵感片段
-

21.3 必备能力

- Markdown 渲染
 - 标签
 - 日期
 - 更新时间
 - 搜索
 - 分类
 - 项目关联
 - 方法论关联
 - 草稿状态
 - 置顶
 - 最近更新
-

21.4 内容关系

一篇 Note 可以关联：

Project
Case Study
Experiment
Thinking Framework
Tag

例如：

Note: Codex 登录问题复盘
关联：
- AI Coding
- Tool Reliability
- Developer Experience

22. Resume App

22.1 作用

为招聘方提供最快路径。

22.2 内容

简历预览
下载 PDF
核心经历摘要
教育经历
项目入口
联系方式

22.3 原则

Resume App 可以直接明确，但不应主导整个网站。

它是 Workspace 中的一个功能，而不是网站全部。

23. Contact App

23.1 内容

- Email
 - GitHub
 - LinkedIn 或其他职业平台
 - 个人社交链接
 - 当前合作意向
 - 简短联系说明
-

23.2 可选能力

- Copy Email
- Open Mail Client
- Download Resume
- External Links
- 简单联系表单

第一阶段不需要后端表单。

24. 第四层：Content Engine

24.1 定义

Content Engine 负责内容的：

- 存储
- 分类
- 标签
- 关联
- 排序
- 搜索
- 状态
- 更新

24.2 推荐目录结构

```
content/  
├── site.config.ts  
├── profile.ts  
├── projects/  
│   ├── fitcheck.mdx  
│   ├── zero-to-one.mdx  
│   └── repo-harness.mdx  
├── cases/  
├── lab/  
├── thinking/  
├── notes/  
├── resume.ts  
├── social.ts  
└── status.ts
```

24.3 Project Schema

```
export type ProjectStatus =  
  | "mature"  
  | "shipped"  
  | "building"  
  | "prototype"  
  | "research"  
  | "archived";  
  
export type Project = {
```

```

id: string;
slug: string;
title: string;
subtitle?: string;

category: string;
status: ProjectStatus;
maturityLevel: number;

problemStatement: string;
oneLiner: string;
personalAngle: string;

role: string[];
capabilities: string[];
tags: string[];

featured: boolean;
order?: number;

startedAt?: string;
updatedAt: string;

cover?: string;
gallery?: string[];

demoUrl?: string;
repositoryUrl?: string;
externalUrl?: string;

relatedCaseIds?: string[];
relatedNoteIds?: string[];
relatedThinkingIds?: string[];
};

```

24.4 Note Schema

```

export type Note = {
  id: string;
  slug: string;
  title: string;
  summary?: string;

  type:
    | "learning"
    | "project-log"
    | "research"
    | "observation"
};

```

```
| "tool-review"  
| "reflection";  
  
tags: string[];  
  
createdAt: string;  
updatedAt: string;  
  
draft: boolean;  
pinned?: boolean;  
  
relatedProjectIds?: string[];  
relatedCaselIds?: string[];  
relatedThinkingIds?: string[];  
};
```

24.5 Thinking Schema

```
export type ThinkingItem = {  
  id: string;  
  slug: string;  
  title: string;  
  
  type:  
    | "framework"  
    | "principle"  
    | "playbook"  
    | "essay"  
    | "methodology";  
  
  problem: string;  
  summary: string;  
  
  tags: string[];  
  steps?: string[];  
  
  relatedProjectIds?: string[];  
  relatedCaselIds?: string[];  
  relatedNotelIds?: string[];  
  
  createdAt: string;  
  updatedAt: string;  
};
```

24.6 Status Schema

```
export type PersonalStatus = {
  currentlyBuilding: {
    title: string;
    description?: string;
    href?: string;
  }[];

  currentlyExploring: string[];

  latestUpdate?: {
    title: string;
    date: string;
    href: string;
  };

  availability?: {
    status: "open" | "selective" | "unavailable";
    description: string;
  };

  lastUpdated: string;
};
```

25. 内容关联模型

整个网站不应该是互相孤立的列表。

应支持：

```
Project
↔ Case Study
↔ Thinking
↔ Notes
↔ Lab Experiment
```

示例：

```
Zero-to-One Product Discovery
|—— Case: 如何避免 AI 过早输出 PRD
|—— Thinking: Stage Gate
|—— Thinking: Controller / Producer / Auditor
```

—— Note: Strict Suite 设计
—— Lab: Baseline A/B 测试

这会让网站表现出真正的“系统性”。

26. 搜索与命令系统

26.1 搜索索引字段

搜索应覆盖：

- 标题
- 摘要
- 标签
- 项目名称
- 方法论名称
- 正文关键字
- App 名称
- 状态

26.2 搜索结果分类

Apps
Projects
Cases
Lab
Thinking
Notes
Actions

26.3 Command Actions

Go to Home
Open Projects
Open Resume
Copy Email
Download Resume
Toggle Language
Reduce Motion
Return to Arrival

27. 导航与路由设计

27.1 推荐路由

```
/
├── /workspace
├── /profile
├── /projects
│   ├── /projects/[slug]
├── /cases
│   ├── /cases/[slug]
├── /lab
│   ├── /lab/[slug]
├── /thinking
│   ├── /thinking/[slug]
├── /notes
│   ├── /notes/[slug]
├── /resume
└── /contact
```

27.2 单页体验与真实路由的关系

视觉上可以表现为 Workspace App 切换，但底层应尽量保留真实 URL。

原因：

- 支持分享
- 支持刷新
- 支持浏览器返回
- 支持搜索引擎
- 支持从外部直接进入详情页
- 支持深链接

27.3 直接进入详情页

用户从搜索引擎或外部链接打开某个项目时：

```
直接打开 Project Detail
+
保留 Workspace Shell
+
显示 Back to Projects
```

不强制播放完整 Arrival 动画。

28. 视觉系统与控制台融合

28.1 Arrival 与 Workspace 共用视觉语言

两种模式必须属于同一个产品。

共用：

- 背景色
- Typography
- Accent Color
- Particle Field
- Glass Material
- Border System
- Motion Curve
- Spacing System

28.2 差异

Arrival

更大字体
更大留白
更强电影感
更少信息
更慢运动

Workspace

更高信息密度
更明确导航
更稳定布局
更弱背景运动
更快速交互反馈

28.3 控制台视觉原则

控制台不应像传统 Dashboard。

保持：

- Editorial Layout
- 大标题

- 非对称布局
- 留白
- 低密度导航
- 深色玻璃
- 柔和粒子
- 弱边框

避免：

- 网格化 KPI 卡片
 - 图表仪表盘
 - 密集侧边栏
 - 数据后台表格
 - 高饱和状态色
 - 操作系统拟物化
-

29. 动效系统

29.1 动效分层

环境动效

- Particle Field
- Orbit
- Mouse Light
- Background Glow

导航动效

- Dock Reveal
- Active App Transition
- Command Palette Open
- Panel Open / Close

内容动效

- Section Reveal
- Card Stagger
- Text Reveal
- Detail Transition

状态动效

- Loading
 - Search
 - Empty State
 - Copy Success
 - Navigation Feedback
-

29.2 Arrival 动效

- Hero Text Reveal
 - Slow Parallax
 - Orbit Rotation
 - Particle Interaction
 - Enter Workspace Transition
-

29.3 Workspace 动效

必须比 Arrival 更快、更克制。

建议：

Panel Transition: 250–450ms
Hover Feedback: 150–250ms
Command Palette: 180–300ms
Card Reveal: 400–700ms

29.4 Reduced Motion

当用户开启减少动态效果时：

- 停止复杂 Parallax
 - 停止 Orbit Rotation
 - 降低粒子数量
 - 取消大范围位移
 - 保留简单淡入
 - 不影响信息访问
-

30. 响应式产品策略

30.1 桌面端

完整支持：

- Arrival
 - Dock
 - Command Palette
 - Wide Workspace Panel
 - 双栏详情
 - 粒子和 Orbit
-

30.2 平板端

调整：

- Dock 缩减
 - Panel 单栏优先
 - 减少粒子数量
 - 减弱 Parallax
 - Command Palette 保留
-

30.3 移动端

不能强行复制桌面控制台。

移动端应转化为：

Personal Workspace App

建议：

- 底部 4-5 个核心导航
 - 更多 App 进入 Menu Sheet
 - 全屏内容页面
 - 简化粒子
 - 关闭复杂鼠标交互
 - Orbit 作为静态或轻微背景
 - Command Palette 转化为搜索入口
-

31. 无障碍要求

必须满足：

- 键盘可访问
 - 焦点状态清晰
 - Command Palette 可键盘操作
 - Dock 图标有文本标签
 - 图片有 alt
 - 色彩对比合格
 - 不只依靠颜色传递状态
 - Reduced Motion
 - 语义化 HTML
 - 正确 Heading 层级
 - Modal 有焦点锁定
 - Esc 可关闭 Command Palette
-

32. 性能要求

32.1 核心要求

- 首屏优先加载
 - 粒子系统不阻塞主线程
 - 移动端自动降级
 - 图片延迟加载
 - 字体尽量本地或系统字体
 - 动效使用 transform / opacity
 - 避免频繁布局计算
 - 避免过早引入重型 Three.js
-

32.2 粒子降级策略

高性能设备

完整粒子 + 连线 + 鼠标交互

中等设备

减少粒子与连接线

移动设备

少量粒子或静态噪声

Reduced Motion

静态背景或极低速粒子

33. 内容与模板解耦

33.1 模板负责

- 视觉
 - 布局
 - 组件
 - 动效
 - 导航
 - 搜索
 - 数据渲染
 - 状态管理
-

33.2 内容负责

- 标题

- 文案
- 项目
- 图片
- 标签
- 状态
- 日期
- 链接
- 关联关系

33.3 禁止

禁止在 JSX 组件中直接写死大量个人内容。

错误：

```
<ProjectCard
  title="FitCheck"
  description="..."
/>
```

正确：

```
projects.map((project) => (
  <ProjectCard key={project.id} project={project} />
));
```

34. 后续内容管理系统

第一阶段不做完整 CMS。

建议演进顺序：

```
Phase 1
TypeScript Config + MDX
```

```
Phase 2
本地可视化内容编辑器
```

```
Phase 3
Git-based CMS
```

Phase 4
独立后台或云端内容系统

34.1 理想更新流程

未来新增笔记：

填写标题
↓
选择类型和标签
↓
输入 Markdown
↓
关联项目
↓
预览
↓
发布
↓
自动更新 Notes、搜索和 Latest Update

35. 推荐技术架构

35.1 第一阶段

Next.js
TypeScript
Tailwind CSS
Framer Motion
Lenis
Canvas Particle System
MDX
Fuse.js 或 MiniSearch
Zustand

35.2 各技术职责

Next.js

- 路由
- 页面
- 静态生成

- SEO
- 部署

TypeScript

- Content Schema
- App State
- 数据约束

Tailwind CSS

- Design Tokens
- 响应式样式
- 组件样式

Framer Motion

- Panel Transition
- Reveal
- Dock
- Command Palette
- Hover Motion

Lenis

- Arrival Smooth Scroll
- 电影化滚动体验

Canvas

- 粒子
- 连线
- 鼠标交互

MDX

- 项目详情
- 案例
- 方法论
- 笔记

Zustand

- Workspace 状态
 - Active App
 - Command Palette
 - 导航历史
-

36. 推荐工程结构

```
clancy-space/
├── app/
│   ├── layout.tsx
│   ├── page.tsx
│   ├── workspace/
│   ├── projects/
│   ├── cases/
│   ├── lab/
│   ├── thinking/
│   ├── notes/
│   ├── resume/
│   └── contact/
├── components/
│   ├── arrival/
│   │   ├── HeroSection.tsx
│   │   ├── IdentityStrip.tsx
│   │   ├── FocusPreview.tsx
│   │   └── EnterWorkspace.tsx
│   ├── workspace/
│   │   ├── WorkspaceShell.tsx
│   │   ├── WorkspaceHeader.tsx
│   │   ├── GlobalDock.tsx
│   │   ├── CommandPalette.tsx
│   │   ├── WorkspaceStatus.tsx
│   │   ├── WorkspacePanel.tsx
│   │   ├── PanelHeader.tsx
│   │   ├── Breadcrumb.tsx
│   │   └── ActiveAppIndicator.tsx
│   ├── apps/
│   │   ├── HomeApp.tsx
│   │   ├── ProfileApp.tsx
│   │   ├── ProjectsApp.tsx
│   │   ├── CasesApp.tsx
│   │   ├── LabApp.tsx
│   │   ├── ThinkingApp.tsx
│   │   ├── NotesApp.tsx
│   │   ├── ResumeApp.tsx
│   │   └── ContactApp.tsx
│   └── motion/
│       ├── ParticleField.tsx
│       ├── OrbitLayer.tsx
│       ├── MouseLight.tsx
│       ├── SmoothScrollProvider.tsx
│       └── WorkspaceTransition.tsx
```

```
|
|
|   └── content/
|       ├── ProjectCard.tsx
|       ├── CaseCard.tsx
|       ├── LabCard.tsx
|       ├── ThinkingCard.tsx
|       └── NoteCard.tsx
|
|   └── ui/
|       ├── GlassButton.tsx
|       ├── StatusPill.tsx
|       ├── AppIcon.tsx
|       ├── SearchResult.tsx
|       ├── Tag.tsx
|       └── EmptyState.tsx
|
└── content/
    ├── site.config.ts
    ├── profile.ts
    ├── status.ts
    ├── projects/
    ├── cases/
    ├── lab/
    ├── thinking/
    └── notes/

└── store/
    └── workspace.store.ts

└── lib/
    ├── content.ts
    ├── search.ts
    ├── motion.ts
    ├── routing.ts
    └── utils.ts

└── styles/
    ├── tokens.css
    ├── motion.css
    └── globals.css

└── public/
    ├── images/
    ├── projects/
    └── resume/
```

37. 实施阶段

Phase 0: 现有初稿审计

在修改代码前完成：

- 当前页面结构分析
- 当前组件分析
- 当前视觉资产分析
- 当前内容耦合分析
- 可复用部分识别
- 需要重构部分识别

输出：

```
Current State Audit
Gap Analysis
Reuse Plan
Refactor Plan
```

不得直接开始大规模编码。

Phase 1: 产品骨架

实现：

- Arrival / Workspace 双模式
- 基础路由
- Workspace State
- Workspace Shell
- Dock
- Panel
- App 切换
- 返回与历史

内容使用占位数据。

验收重点：

网站已经不再只是 Landing Page。

Phase 2: Arrival Experience

实现：

- Hero

- Identity
- Focus Strip
- Particle Field
- Orbit
- Mouse Light
- Enter Workspace Transition

验收重点：

第一屏优雅、清晰、不像产品官网。

Phase 3: Workspace Navigation

实现：

- Global Dock
- Workspace Header
- Command Palette
- 搜索
- Active App
- Breadcrumb
- Navigation History

验收重点：

Workspace 可以稳定探索，不依赖页面滚动。

Phase 4: Content Apps

实现顺序：

1. Home
 2. Projects
 3. Project Detail
 4. Thinking
 5. Notes
 6. Cases
 7. Lab
 8. Profile
 9. Resume
 10. Contact
-

Phase 5: Content Engine

实现：

- MDX
 - Schema
 - Tags
 - Maturity
 - Related Content
 - Latest Update
 - Search Index
 - Sorting
 - Filtering
-

Phase 6: 真实内容接入

接入：

- 项目
 - 方法论
 - 笔记
 - 经历
 - 链接
 - 简历
 - 当前状态
-

Phase 7: 内容编辑能力

实现：

- 内容表单
 - Markdown 编辑
 - 标签选择
 - 项目关联
 - 预览
 - 发布流程
-

Phase 8: 高级视觉升级

基础体验稳定后再考虑：

- Three.js
- Shader
- WebGL 粒子
- 复杂 Depth
- Project Node Network

- Post Processing
-

38. 产品验收标准

38.1 定位验收

用户在 5 秒内可以回答：

- 这是谁的网站
 - 他做什么
 - 他关注什么
 - 我可以在这里看什么
-

38.2 架构验收

必须同时存在：

- Arrival Layer
- Workspace Shell
- Content Apps
- Content Engine

不能只有 Hero 和 Section。

38.3 控制台验收

必须支持：

- App 切换
 - 当前 App 状态
 - 内容打开
 - 返回
 - 搜索或命令
 - Workspace 导航
 - 真实 URL 或可恢复状态
-

38.4 内容验收

必须支持：

- 项目成熟度
- 项目详情
- 方法论
- 笔记

- 内容关联
 - 最近更新
 - 当前状态
-

38.5 视觉验收

必须：

- 保留 V3 / V4 的视觉方向
 - 深石墨、暖白、鼠尾草绿
 - 粒子与 Orbit 克制
 - Workspace 不像后台
 - 不像 macOS 复刻
 - 不像 SaaS Dashboard
 - 不像传统简历模板
-

38.6 动效验收

- Arrival 有电影感
 - Workspace 反馈快速
 - 转场自然
 - 动效不影响阅读
 - Reduced Motion 可用
 - 移动端自动降级
-

38.7 工程验收

- 内容与组件解耦
 - TypeScript Schema 完整
 - App 可独立维护
 - 路由可分享
 - 状态可恢复
 - 移动端可用
 - 性能稳定
 - 可部署
-

39. 风险与防偏离约束

风险 1：重新退化成 Landing Page

判断信号：

- 只有长页面 Section
- Dock 只是锚点导航

- 没有 App 状态
- 没有内容打开模型
- 没有 Command Palette

解决：

Workspace 必须是产品状态，而不是视觉装饰。

风险 2：做成传统 Dashboard

判断信号：

- 左侧密集菜单
- 多个 KPI 卡片
- 图表和数据
- 后台管理布局

解决：

保持 Editorial Workspace，而非 Admin Dashboard。

风险 3：过度模仿 macOS

判断信号：

- 红黄绿窗口按钮
- 彩色 App 图标
- 自由拖拽窗口
- 桌面壁纸与文件夹

解决：

借用工作空间交互，不复制操作系统外观。

风险 4：动效压过内容

判断信号：

- 进入动画时间过长
- 粒子过亮
- 滚动被强制控制
- 用户难以快速找到项目

解决：

Motion 必须服从信息访问效率。

风险 5：一次实现范围过大

解决：

- 先做 Shell
- 再做两个 App
- 再扩展内容
- WebGL 最后做

40. 给 Antigravity 的执行要求

40.1 当前任务

请先阅读并理解本文档。

然后完成以下输出，不要立即修改代码：

1. 对当前初稿进行架构审计。
2. 判断现有代码哪些部分可以保留。
3. 对照本文档列出缺失项。
4. 输出新的产品组件树。
5. 输出状态模型。
6. 输出路由模型。
7. 输出内容 Schema。
8. 输出分阶段实施计划。
9. 输出每阶段验收标准。
10. 等待确认后再开始编码。

40.2 必须保留

- V3 / V4 视觉方向
- Cinematic Arrival
- 粒子系统
- Orbit
- Editorial Typography
- 深石墨 + 暖白 + 鼠尾草绿
- 内容与模板解耦

40.3 必须新增

- Arrival / Workspace 双模式
- Workspace Shell
- Global Dock

- Command Palette
- Workspace Panel
- Active App State
- Content Apps
- 项目成熟度分层
- Notes 系统
- 内容关联模型
- 当前状态系统

40.4 禁止直接执行的错误方向

不要只新增一个 Dock。
不要把 Dock 做成页面锚点。
不要只新增几个玻璃卡片。
不要把控制台做成 Dashboard。
不要做 macOS 复刻。
不要直接填充全部真实内容。
不要立即引入复杂 Three.js。
不要在没有状态模型和路由模型前重构页面。

41. 最终产品表达

Clancy Space 的最终结构应是：

```
Cinematic Arrival
+
Clear Personal Identity
+
Editorial Workspace
+
Content Apps
+
Command & Search
+
Living Content System
```

它不是：

漂亮的个人主页

而是：

一个可以持续承载项目、案例、方法论、实验、笔记与个人状态的 Editorial Personal Operating System。

最终目标不是让访问者只觉得：

“这个网站很好看。”

而是让访问者形成更完整的判断：

“这个人对产品、系统、内容和体验有完整的结构化理解，而这个网站本身就是这种能力的证明。”